



Security Audit

Onramp (DeFi)

Table of Contents

Executive Summary	4
Project Context	4
Audit Scope	7
Security Rating	9
Intended Smart Contract Functions	10
Code Quality	13
Audit Resources	13
Dependencies	13
Severity Definitions	14
Status Definitions	15
Audit Findings	16
Centralisation	36
Conclusion	37
Our Methodology	38
Disclaimers	40
About Hashlock	41

CAUTION

THIS DOCUMENT IS A SECURITY AUDIT REPORT AND MAY CONTAIN CONFIDENTIAL INFORMATION. THIS INCLUDES IDENTIFIED VULNERABILITIES AND MALICIOUS CODE WHICH COULD BE USED TO COMPROMISE THE PROJECT. THIS DOCUMENT SHOULD ONLY BE FOR INTERNAL USE UNTIL ISSUES ARE RESOLVED. ONCE VULNERABILITIES ARE REMEDIATED, THIS REPORT CAN BE MADE PUBLIC. THE CONTENT OF THIS REPORT IS OWNED BY HASHLOCK PTY LTD FOR USE OF THE CLIENT.

Executive Summary

The Onramp team partnered with Hashlock to conduct a security audit of their smart contracts. Hashlock manually and proactively reviewed the code in order to ensure the project's team and community that the deployed contracts are secure.

Project Context

Hann Finance is positioning itself as a stability-first DeFi platform built for the KAIA ecosystem, targeting users familiar with web3 liquidity-markets ("Kaia degens"). The website invites users to join a waitlist and hints that its testnet is already live. It emphasises simplicity and robustness, promising a next-generation source of liquidity with minimal complexity.

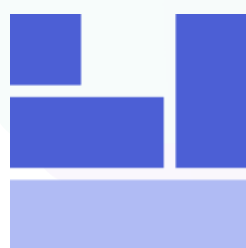
Project Name: Hann Finance

Project Type: Defi

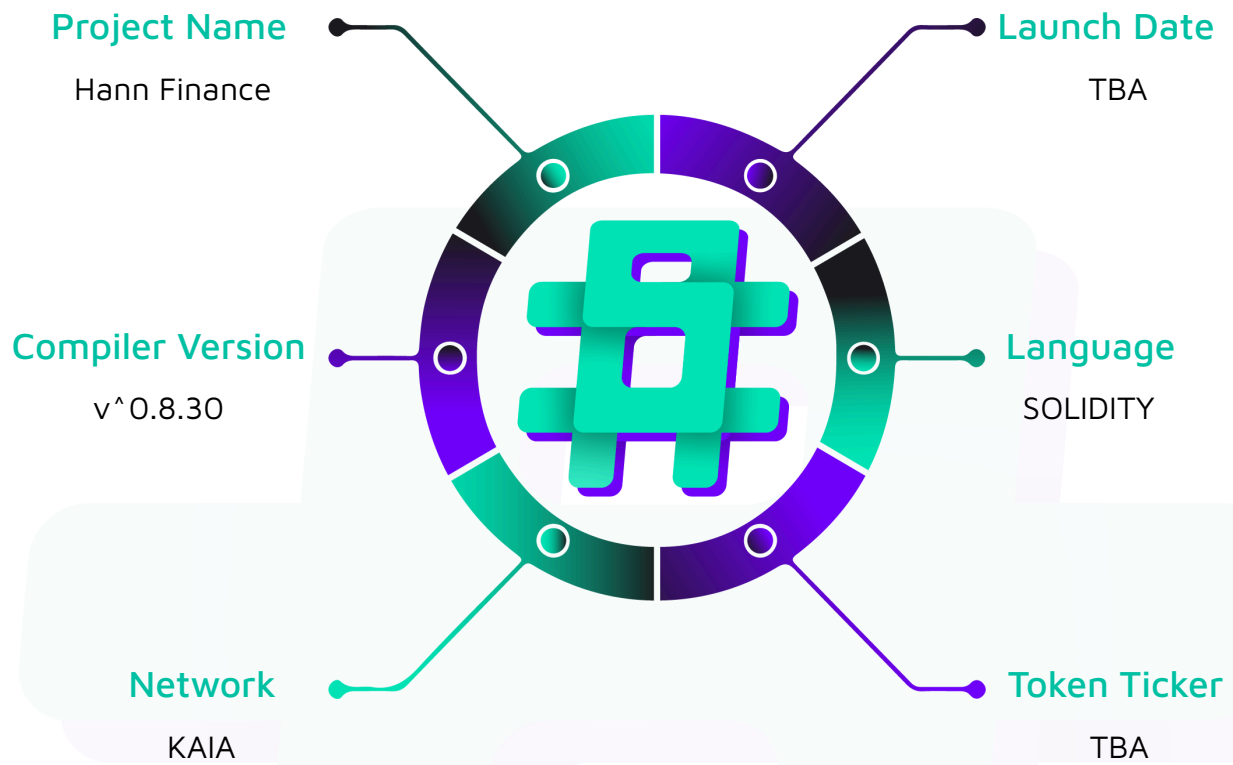
Compiler Version: ^0.8.30

Website: <https://hann.finance>

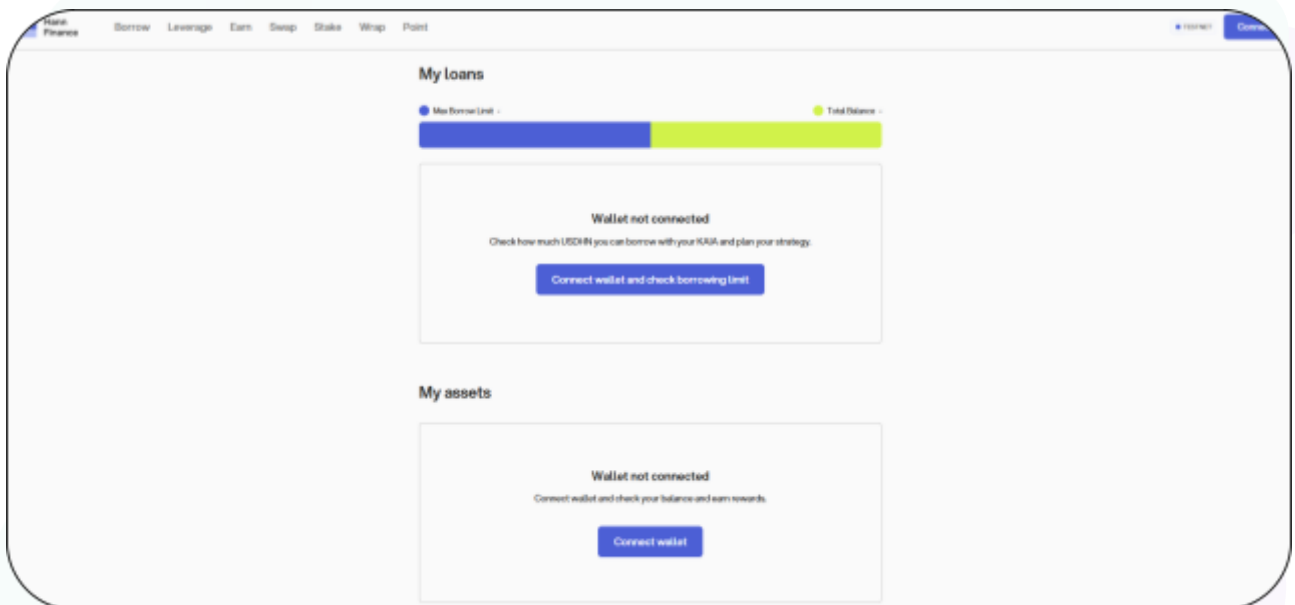
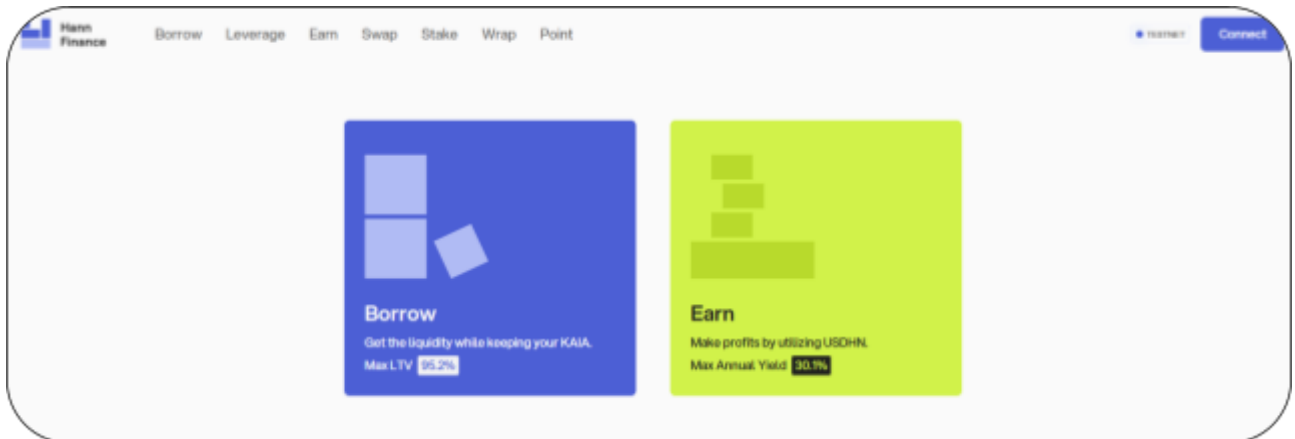
Logo:



Hann Finance

Visualised Context:

Project Visuals:



Audit Scope

We at Hashlock audited the solidity code within the Hann Finance project. The scope of work included a comprehensive review of the smart contracts listed below. We tested the smart contracts to check for their security and efficiency. These tests were undertaken primarily through manual line-by-line analysis and were supported by software-assisted testing.

Description	Hann Finance Smart Contracts
Platform	KAIA / Solidity
Audit Date	November, 2025
GitHub link3	https://github.com/Hann-Finance/lst-core
Contract 1	adapters/ICnStakingV3.sol
Contract 2	adapters/IKIP163.sol
Contract 3	adapters/IPublicDelegation.sol
Contract 4	adapters/Vault4626Adapter.sol
Contract 5	core/NodeControllerLib.sol
Contract 6	core/RouterLib.sol
Contract 7	governance/Timelock.sol
Contract 8	token/LSTToken.sol
Contract 9	vault/LSTVaultAdmin.sol
Contract 10	vault/LSTVaultAdminModule.sol
Contract 11	vault/LSTVaultBase.sol
Contract 12	vault/LSTVaultClaim.sol
Contract 13	vault/LSTVaultDeposit.sol
Contract 14	vault/LSTVaultStorage.sol
Contract 15	vault/LSTVaultUnstake.sol
Contract 16	vault/LSTVaultView.sol

Contract 17	views/LSTVaultViewHelper.sol
Contract 18	views/LSTVault.sol
Audited GitHub Commit Hash	4a0e87326f3c44cdda00bf4376ac9c366585bf60
Fix Review GitHub Commit Hash	d870a633b9bbe91dc7e391f2d2e22a2bc65fae61

NOTE: After applying the fixes for the reported findings, the team introduced a new set of changes to the lst-core codebase with defensive hardening, best-practice alignment, and design clarity improvements.

Commit Hash: [1f4be015036a7155f1bd27a0927fcb9ceb92cad0](#)

Patch Details: [audit_patch_lst_v3.md](#)

Security Rating

After Hashlock's Audit, we found the smart contracts to be **"Secure"**. The contracts all follow simple logic, with correct and detailed ordering. They use a series of interfaces, and the protocol uses a list of Open Zeppelin contracts.



The 'Hashlocked' rating is reserved for projects that ensure ongoing security via bug bounty programs or on-chain monitoring technology.

All issues uncovered during automated and manual analysis were meticulously reviewed and applicable vulnerabilities are presented in the [Audit Findings](#) section. The list of audited assets is presented in the [Audit Scope](#) section, and the project's contract functionality is presented in the [Intended Smart Contract Functions](#) section.

All vulnerabilities initially identified have now been resolved and acknowledged.

Hashlock found:

1 High severity vulnerabilities

3 Medium severity vulnerabilities

5 Low severity vulnerabilities

4 QA

Caution: Hashlock's audits do not guarantee a project's success or ethics, and are not liable or responsible for security. Always conduct independent research about any project before interacting.

Intended Smart Contract Functions

Claimed Behaviour	Actual Behaviour
LST-Core • Modular Liquid Staking Vault: Implements a sophisticated, upgradeable vault for liquid staking. It accepts native asset deposits and in return, mints a yield-bearing Liquid Staking Token (LST). • Delegated Staking and Node Management: The vault delegates deposited assets across a distributed network of staking nodes. It includes robust administrative controls for managing the lifecycle of these nodes, such as registration, pausing, and adjusting delegation weights. • Timelock Governance: Critical administrative functions and contract upgrades are managed by a Timelock contract. This enforces a mandatory time delay on all changes, providing transparency and security for users. • Advanced Reward and Fee Mechanics: Contains dedicated modules for handling the accounting of staking rewards, processing user claims, and managing protocol fees. • External Protocol Integration: Designed to integrate with external staking systems, such as Kaia's Public Delegation and CnStaking protocols, to perform its core staking functions.	Contract achieves this functionality.

Code Quality

This audit scope involves the smart contracts of the Hann Finance project, as outlined in the Audit Scope section. All contracts, libraries, and interfaces mostly follow standard best practices and to help avoid unnecessary complexity that increases the likelihood of exploitation, however, some refactoring were recommended to optimize security measures.

The code is very well commented on and closely follows best practice nat-spec styling. All comments are correctly aligned with code functionality.

Audit Resources

We were given the Hann Finance project smart contract code in the form of GitHub access.

As mentioned above, code parts are well commented. The logic is straightforward, and therefore it is easy to quickly comprehend the programming flow as well as the complex code logic. The comments are helpful in providing an understanding of the protocol's overall architecture.

Dependencies

As per our observation, the libraries used in this smart contracts infrastructure are based on well-known industry standard open source projects.

Apart from libraries, its functions are used in external smart contract calls.

Severity Definitions

The severity levels assigned to findings represent a comprehensive evaluation of both their potential impact and the likelihood of occurrence within the system. These categorizations are established based on Hashlock's professional standards and expertise, incorporating both industry best practices and our discretion as security auditors. This ensures a tailored assessment that reflects the specific context and risk profile of each finding.

Significance	Description
High	High-severity vulnerabilities can result in loss of funds, asset loss, access denial, and other critical issues that will result in the direct loss of funds and control by the owners and community.
Medium	Medium-level difficulties should be solved before deployment, but won't result in loss of funds.
Low	Low-level vulnerabilities are areas that lack best practices that may cause small complications in the future.
Gas	Gas Optimisations, issues, and inefficiencies.
QA	Quality Assurance (QA) findings are informational and don't impact functionality. Supports clients improve the clarity, maintainability, or overall structure of the code.

Status Definitions

Each identified security finding is assigned a status that reflects its current stage of remediation or acknowledgment. The status provides clarity on the handling of the issue and ensures transparency in the auditing process. The statuses are as follows:

Significance	Description
Resolved	The identified vulnerability has been fully mitigated either through the implementation of the recommended solution proposed by Hashlock or through an alternative client-provided solution that demonstrably addresses the issue.
Acknowledged	The client has formally recognized the vulnerability but has chosen not to address it due to the high cost or complexity of remediation. This status is acceptable for medium and low-severity findings after internal review and agreement. However, all high-severity findings must be resolved without exception.
Unresolved	The finding remains neither remediated nor formally acknowledged by the client, leaving the vulnerability unaddressed.

Audit Findings

High

[H-01] LSTVaultAccountingModule#flushSurplusImmediateFromVault -

Escrow reserved funds incorrectly routed to PDs

Description

The `flushSurplusImmediateFromVault()` function uses `address(this).balance` instead of `_liquidBalance()`, causing escrow reserved funds to be incorrectly routed instead of remaining available for user withdrawals.

Vulnerability Details

The `flushSurplusImmediateFromVault()` function uses `address(this).balance` to determine the amount of funds to route to protocol delegations:

```
function flushSurplusImmediateFromVault(uint256 minTrigger) external payable {  
  
    bool lockSet = _takeViewLockInternal();  
  
    if (!paused()) {  
  
        uint256 balance = address(this).balance; // includes escrow funds  
  
        if (balance >= minTrigger) {  
  
            uint256 rawAssets = _totalAssetsRawInternal();  
  
            uint256 sPrime = rawAssets > balance ? rawAssets - balance : 0;  
  
            // ... routes balance to PDs  
  
        }  
  
    }  
  
    // ...  
  
}
```

However, `address(this).balance` includes escrow reserved funds that are held for users who could not receive payouts. These funds should be excluded from surplus routing operations.

The contract provides a `_liquidBalance()` function that correctly excludes escrow reserved funds. This function is correctly used in `reinvestSurplusManagedFromVault()`, demonstrating the proper pattern for excluding escrow funds from routing operations.

Impact

Escrow reserved user funds are incorrectly routed to protocol delegations, making them unavailable for user withdrawals and resulting in loss of user funds.

Recommendation

Replace `address(this).balance` with `_liquidBalance()` to exclude escrow reserved funds from surplus routing.

Status

Resolved

Medium

[M-01] LSTVaultPlanModule#depositToExact - Share limit bypass

Description

The `maxShareBps` validation in the `depositToExact` function can be bypassed by using different receiver addresses or transferring LST tokens to different addresses before making new deposits, allowing whitelisted accounts to accumulate unlimited vault shares beyond their configured limits.

Vulnerability Details

The share ownership limit validation in `depositToExact` only checks the receiver's current token balance at the time of deposit:

```
function depositToExact(uint256 nodeId, address receiver, address controller, uint256
minShares) {

    // ...

    DirectedConfig storage cfg = directedWhitelist[msg.sender];

    // ...

    if (cfg.maxShareBps > 0 && ts > 0) {

        uint256 newTotal = ts + shares;

        uint256 postBalance = _shareBalanceOfInternal(receiver) + shares;

        uint256 postShareBps = FPMX.mulDivUp(postBalance, BPS_DENOM, newTotal);

        if (postShareBps > cfg.maxShareBps) revert DirectedShareLimit();

    }

    // ...

}

function _shareBalanceOfInternal(address account) internal view returns (uint256) {

    address shareToken = token;

    if (shareToken == address(0)) {
```

```

        return 0;
    }

    return LSTToken(shareToken).balanceOf(account);
}

```

The validation uses `_shareBalanceOfInternal(receiver)` which only returns the current balance of the receiver address, not the cumulative shares owned by the whitelisted account (`msg.sender`) across all addresses. Since LST tokens are standard ERC20 tokens with no transfer restrictions, a whitelisted account can exploit this in two ways:

1. Using different receiver addresses: Make multiple `depositToExact` calls with different receiver addresses, each having 0 balance, allowing unlimited share accumulation
2. Transferring tokens first: Transfer existing LST tokens to another address before making a new deposit with a different receiver address

In both cases, the validation passes because `postBalance` only considers the receiver's current balance, not the whitelisted account's cumulative share ownership across all addresses.

The `DirectedConfig` struct tracks `usedAmount` (cumulative assets) but not cumulative shares, making the share limit validation fundamentally flawed.

Impact

Whitelisted accounts can bypass share ownership limits and accumulate unlimited vault shares, leading to concentration risk

Recommendation

Implement cumulative share tracking similar to the existing `usedAmount` pattern by adding a `usedShareAmount` field to the `DirectedConfig` struct and validating against cumulative shares minted to the whitelisted account rather than the receiver's current balance.

```

if (cfg.maxShareBps > 0 && ts > 0) {
    uint256 newTotal = ts + shares;

```

```
uint256 newUsedShares = cfg.usedShareAmount + shares;

uint256 postShareBps = FPMX.mulDivUp(newUsedShares, BPS_DENOM, newTotal);

if (postShareBps > cfg.maxShareBps) revert DirectedShareLimit();

cfg.usedShareAmount = newUsedShares;

}
```

Status

Resolved

[M-02] Vault4626Adapter#requestRedeem - Partial share burn breaks ERC-7540 compliance

Description

The `requestRedeem()` function in `Vault4626Adapter` allows partial burning of shares when not all assets can be routed to withdrawal nodes, violating the ERC-7540 standard requirement that the full requested share amount must be burned atomically at request time or the transaction must revert.

Vulnerability Details

Under ERC-7540, the `requestRedeem()` function operates on a specified shares amount and must fully accept and burn those shares at request time. If it cannot process the full amount, it must revert.

The current implementation flow is as follows:

1. `Vault4626Adapter.requestRedeem()` calls `_callRequestRedeem()` which invokes the vault's `requestRedeem()` function.
2. The vault's `requestRedeem()` in `VaultUnstakeEntryModule.requestRedeem()` calls `_requestUnstakeShares(owner, shares)`.
3. This eventually calls `_requestUnstakeSharesInternal()` in `LSTVaultOpsUnstakeLogic`.
4. The function calls `_executeUnstakePlanInternal()` which can return `totalRouted < plannedTotal` when some PD withdrawal tickets fail to open.
5. When `totalRouted < plannedTotal`, the function calculates `sharesToBurn` based on the actual `executedAssets`, which results in `sharesToBurn < shares`.
6. The function then burns only `sharesToBurn` amount, leaving some shares unburned.
7. Finally, `Vault4626Adapter.requestRedeem()` incorrectly emits the `RedeemRequest` event with the full `shares` amount, even though fewer shares were actually burned.

Impact

Users requesting to redeem shares may only have a portion burned, leaving their remaining shares in their account unredeemed, requiring them to make additional requests to complete the redemption. While this does not result in loss of funds (the unburned shares remain with the user), it creates a poor user experience and breaks compliance with the ERC-7540 standard. Additionally, the event emission misrepresents the actual state, leading to inaccurate off-chain tracking and potential integration failures.

Recommendation

Ensure the full requested shares will be burned at request time and otherwise revert. As an alternative, consider documenting the current behavior and update the event emission to reflect the actual shares burned rather than the requested amount.

Status

Acknowledged

[M-03] LSTVaultFeeModule#chargePerformanceFee - Deposits after market losses reset High-Water mark, causing unfair performance fees

Description

The high-water mark mechanism is designed to protect users from paying performance fees until their losses are fully recovered. However, when a large deposit occurs after a market loss, the system resets the high-water mark to the current (lower) Price Per Share (PPS) instead of maintaining the original high-water mark. This allows performance fees to be charged on market recovery even though users who invested before the loss haven't yet recovered their original position, violating the intended protection mechanism.

Vulnerability Details

The high-water mark reset issue is related to code in two functions: `LSTVaultAccountingModule._maybeUpdatePerformanceSnapshotInternal()` which updates the snapshot after deposits without PPS validation, and `LSTVaultFeeModule.chargePerformanceFee()` which calculates fees during recovery without verifying PPS has improved.

In `LSTVaultAccountingModule._maybeUpdatePerformanceSnapshotInternal()`, the snapshot is updated when `assets > snapAssets` without checking if PPS has improved:

```
// LSTVaultAccountingModule._maybeUpdatePerformanceSnapshotInternal()
if (assets > snapAssets) {
    _setPerformanceSnapshotInternal(assets, supply); // Updates without PPS check
    return;}

```

When `assets > snapAssets`, the snapshot is immediately updated without validating `currentPps >= snapPps`. The PPS check only occurs when `supply < snapSupply` (redemptions), not when deposits increase total assets.

In `LSTVaultFeeModule.chargePerformanceFee()`, fees are calculated from asset growth without PPS validation:

```
// LSTVaultFeeModule.chargePerformanceFee()

uint256 profit = currentAssets - snapAssets; // No PPS validation

uint256 feeAssets = FPMX.mulDivDown(profit, performanceFeeBps, BPS_DENOM);

// ... fee calculation and minting ...

_setSnapshot(currentAssets, supply);
```

When `currentAssets > snapAssets`, profit is calculated as a simple delta and fees are charged without verifying PPS has recovered. The PPS check only applies when `currentAssets <= snapAssets`, which is the opposite scenario.

Consider the following scenario:

1. Market loss drops PPS (e.g., 1.0 → 0.5) while snapshot remains at original high-water mark
2. Large deposit increases assets above `snapAssets` but PPS unchanged; snapshot gets reset to lower PPS without validation
3. Market recovery triggers `chargePerformanceFee()` which calculates profit against the lowered snapshot, charging fees even though PPS is still below the original high-water mark.

Impact

Users who invested before a market crash can be charged performance fees even though they haven't recovered their losses yet. This is unfair to users and violates the intended high-water mark protection.

Recommendation

Add validation so that the performance snapshot is correctly updated, and performance fees are only charged, when the current PPS is at least as high as the stored snapshot PPS, preserving the intended high-water mark behavior.

Status

Resolved

Low

[L-01] Vault4626Adapter#previewRedeem,previewWithdraw - Not compliant with ERC-7540

Description

The `previewRedeem` and `previewWithdraw` functions in `Vault4626Adapter` do not revert as required by `ERC-7540` for asynchronous redeem vaults, violating the standard specification.

Vulnerability Details

According to the `ERC-7540` specification, asynchronous redeem vaults must override the `ERC-4626` specification as follows: "`previewRedeem` and `previewWithdraw` MUST revert for all callers and inputs."

The `Vault4626Adapter` contract implements `IERC7540Vault`, indicating it is an asynchronous redeem vault. However, the `previewRedeem` and `previewWithdraw` functions in the base contract `Vault4626AdapterBase` do not revert.

Impact

The implementation violates the `ERC-7540` standard specification, which may cause integration issues with protocols and tools that expect these functions to revert for asynchronous redeem vaults. Users and integrators may receive misleading preview values that do not accurately represent the actual assets they will receive from asynchronous redemption operations.

Recommendation

Modify `previewRedeem` and `previewWithdraw` to always revert.

Status

Resolved

[L-02] Vault4626Adapter#requestRedeemAssets - Incorrect RedeemRequest event parameter

Description

The `requestRedeemAssets()` function emits the `RedeemRequest` event with `assets` as the last parameter instead of `shares`, which violates the EIP-7540 standard specification.

Vulnerability Details

According to EIP-7540, the `RedeemRequest` event signature is defined as:

```
event RedeemRequest(  
    address indexed controller,  
    address indexed owner,  
    uint256 indexed requestId,  
    address sender,  
    uint256 shares  
);
```

In `Vault4626Adapter`, the `requestRedeem()` function correctly emits the event with `shares`:

```
function requestRedeem(uint256 shares, address controller, address owner) external  
returns (uint256 requestId) {  
    // ... validation logic ...  
    requestId = _callRequestRedeem(shares, ownerResolved);  
    _registerRequest(requestId, controllerResolved, ownerResolved);  
    emit RedeemRequest(controllerResolved, ownerResolved, requestId, msg.sender, shares);  
}
```

However, the `requestRedeemAssets()`, which is not part of the EIP-7540 standard, incorrectly emits the event with `assets` instead of `shares`:

```
function requestRedeemAssets(uint256 assets, address controller, address owner)

    external

    returns (uint256 requestId)

{

    // ... validation logic ...

    requestId = _callRequestRedeemAssets(assets, ownerResolved);

    _registerRequest(requestId, controllerResolved, ownerResolved);

    emit RedeemRequest(controllerResolved, ownerResolved, requestId, msg.sender, assets);

}
```

This is also inconsistent with the `VaultUnstakeEntryModule.requestRedeemAssets()` function, which correctly emits the `RedeemRequest` event with shares.

Impact

This can cause issues for external off-chain integrations, event indexers, and frontend applications that rely on the EIP-7540 event structure, potentially leading to incorrect data interpretation in external tooling.

Recommendation

Consider calculate the correct number of shares to emit the `RedeemRequest` event with `shares` or use a different event parameter name with the assets value.

Status

Resolved

[L-03] Vault4626Adapter - Missing ERC-20 Interface Implementation

Description

The `Vault4626Adapter` contract implements the ERC-7535 (native ERC-4626) interface but does not implement the required ERC-20 functions, making it non-compliant with the ERC-4626 specification which mandates ERC-20 implementation.

Vulnerability Details

According to the ERC-4626 specification, "All EIP-4626 tokenized Vaults MUST implement EIP-20 to represent shares." Since ERC-7535 is an extension of ERC-4626, the same requirement applies.

The `Vault4626Adapter` contract implements `IERC7535NativeVault` which extends ERC-4626, but it does not implement the following required ERC-20 functions:

- `totalSupply()` - to return the total supply of vault shares
- `balanceOf(address account)` - to return the share balance of an account
- `transfer(address to, uint256 amount)` - to transfer shares
- `transferFrom(address from, address to, uint256 amount)` - to transfer shares on behalf of another account
- `approve(address spender, uint256 amount)` - to approve spending of shares
- `allowance(address owner, address spender)` - to check spending allowance

The contract appears to be designed as an adapter that facilitates the interactions with the `LSTVault` and not intended to be minting or burning a new token besides the `LSTToken`, but for full ERC-4626/ERC-7535 compliance, it should implement ERC-20 functions. (potentially delegating to the vault's share token) or explicitly documenting that it's a partial implementation.

Impact

This non-compliance means that the `Vault4626Adapter` cannot be used with standard ERC-4626 tooling and integrations that expect ERC-20 functions to be available, potentially breaking compatibility with aggregators, DeFi protocols, and frontend applications that rely on the full ERC-4626 interface.

Recommendation

Consider the following options:

1. If the adapter is not intended to be minting or burning a new token besides the LSTToken, consider explicitly documenting this limitation about ERC-7535/ERC-4626 compliance.
2. If the adapter is intended to be fully compliant with ERC-7535/ERC-4626, implement the required ERC-20 functions by implementing ERC-20 functionality directly in the adapter by maintaining share balances and emitting the required events.

Status

Resolved

[L-04] LSTVaultAdminModule#setStrictAccountingFromVault - setStrictAccountingFromVault enforces pause+admin but is unused and MANAGER_ROLE can disable strict accounting without pause

Description

The vault-only method that enforces pause and DEFAULT_ADMIN_ROLE when disabling strict accounting is unused, while the active toggle lets MANAGER_ROLE switch it without pause; comments suggest a guard that isn't applied.

Vulnerability Details

Design intent indicates disabling strict accounting should require pause and admin via a vault-only path, but no code routes to it. The effective toggle is exposed in the entry module to MANAGER_ROLE without a pause requirement, making the guard dead and documentation misleading.

Impact

Operational policy mismatch may allow managers to disable strict accounting without pausing, contrary to stated intent.

Recommendation

Decide the intended policy and align the codebase. Either remove the unused vault-only method and update comments to reflect manager-controlled toggling without pause (current behavior), or route toggling through the vault-only path and enforce pause plus DEFAULT_ADMIN_ROLE. Update tests and documentation accordingly

Status

Resolved

[L-05] LSTVaultAccountingModule# - Missing delegate call guard on accounting module payable functions

Description

Several external payable functions in `LSTVaultAccountingModule` lack `_requireVaultDelegatecall()`, allowing direct calls and KAIA transfers to the module contract, which is inconsistent with other modules.

Vulnerability Details

Functions like `refreshNavSnapshotFromVault`, `updatePerformanceSnapshot FromVault`, `flushSurplusImmediateFromVault`, `chargePerformanceFeeFromVault`, `reinvestSurplusManagedFromVault`, `maybeUpdatePerformanceSnapshotFromVault`, `syncAccountedBalanceFromVault`, and `setPerformanceSnapshotFromVault` are external and payable without enforcing `delegatecall`. Direct calls mutate the module's own storage and can store KAIA on the module, not the vault.

Impact

No impact on vault funds or accounting because direct calls don't touch vault storage, but it can accumulate KAIA on the module, emit misleading events..

Recommendation

Add `_requireVaultDelegatecall()` to all external functions in `LSTVaultAccountingModule`, drop payable where not needed, and consider a reverting receive/fallback to prevent accidental KAIA transfers.

Status

Resolved

QA

[Q-01] LSTVaultOpsModule#_routeDepositInternal - Unreachable zero-weight fallback in deposit routing

Description

A fallback assignment intended to handle zero-weight nodes in deposit waterfilling is never executed and has no effect.

Vulnerability Details

The code sets caps and the associated weight only when `target > stake`. With `weightBps = 0`, target is 0, so the condition never holds and the fallback assignment for zero-weight nodes is unreachable. Subsequent redistribution and fallback paths do not consume this value.

Impact

No functional or security impact; routing remains correct and zero-weight nodes receive no allocation. The dead code can mislead readers.

Recommendation

Remove the zero-weight fallback assignment (or move it outside the unreachable branch if a future policy intends zero-weight participation).

Status

Resolved

[Q-02] LSTCore - Missing NatSpec documentation in some functions

Description

Most of the contracts functions lack NatSpec documentation that explains their intended functionality. This makes the code harder to understand, maintain, and audit.

Vulnerability Details

All contracts have some NatSpec documentation for it but it is missing in most functions explaining function purposes, parameters, return values and implementation details.

Impact

Lack of documentation makes the code harder to understand, maintain, and integrate with, while also reducing audit efficiency.

Recommendation

Add comprehensive NatSpec documentation to all missing functions.

Status

Resolved

[Q-03] LSTCore - Magic numbers should be replaced with named constants**Description**

Multiple contracts extensively use magic numbers like `1e18` and `32` throughout the codebase instead of defining them as named constants.

Vulnerability Details

The contracts use magic numbers in multiple places for calculations and comparisons like `1e18` and `32`.

Impact

While this doesn't pose a security risk, it significantly impacts code quality and developer experience. Magic numbers make the code harder to maintain and increase the likelihood of bugs and makes future modifications more error-prone.

Recommendation

Define named constants at the contract level and replace all magic numbers.

Status

Resolved

[Q-04] LSTCore#Vault4626Adapter, LSTVaultPlanModule - Unused imports and constants

Description

Vault4626Adapter.sol and LSTVaultPlanModule.sol contain unused imports that increase code complexity without providing any functionality.

Vulnerability Details

The following unused items were identified:

Vault4626Adapter.sol: The constant VAULT_SET_OPERATOR_SELECTOR is defined but never used in the contract.

LSTVaultPlanModule.sol: The VaultConstants and IPublicDelegation imports are unused.

Impact

While these unused items do not pose a security risk, they increase code complexity.

Recommendation

Remove the unused imports and constants.

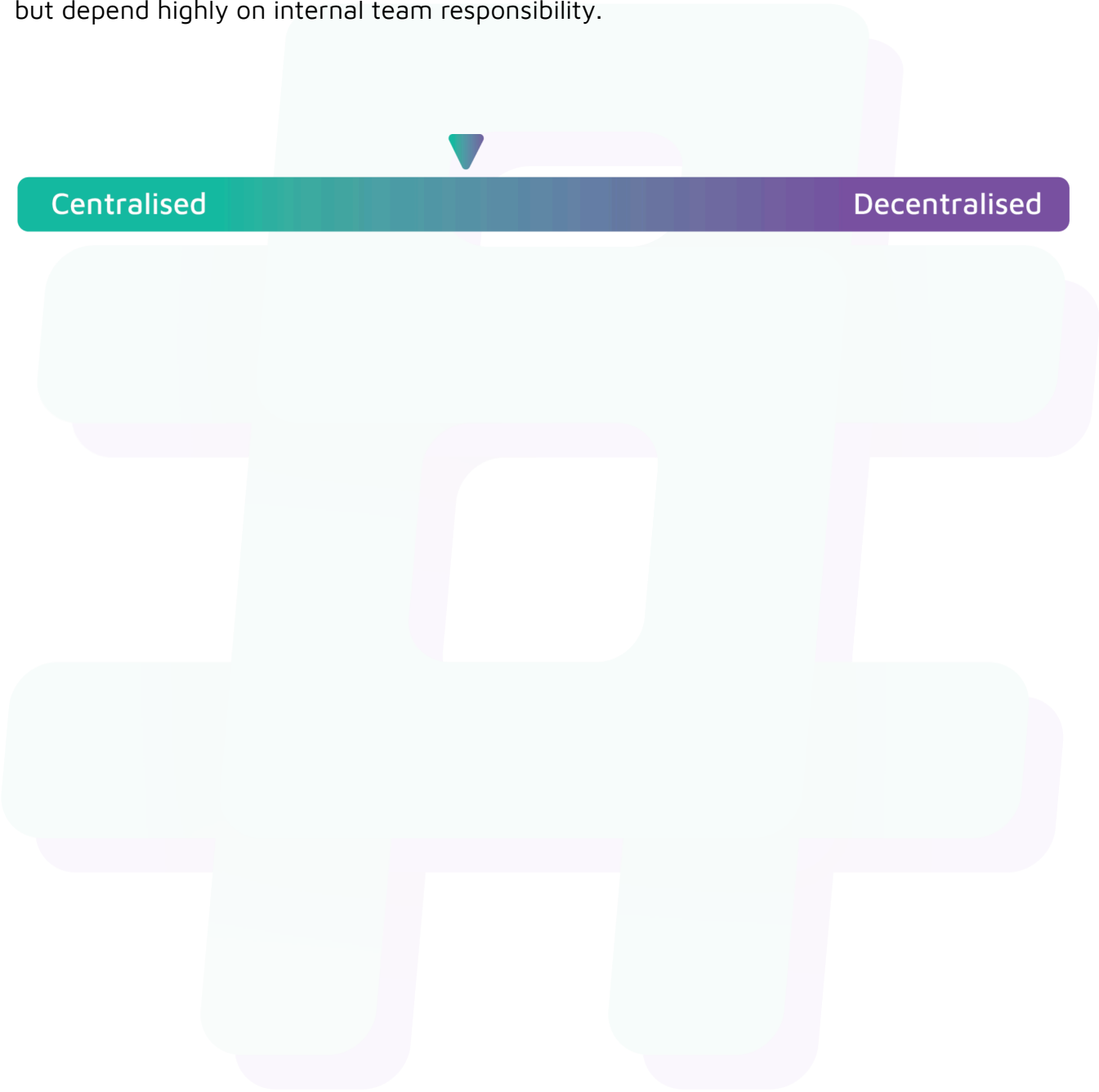
Status

Resolved

Centralisation

The Hann Finance project values security and utility over decentralisation.

The owner executable functions within the protocol increase security and functionality but depend highly on internal team responsibility.



Centralised

Decentralised

Conclusion

After Hashlock's analysis, the Hann Finance project seems to have a sound and well-tested code base, now that our vulnerability findings have been resolved and acknowledged. Overall, most of the code is correctly ordered and follows industry best practices. The code is well commented on as well. To the best of our ability, Hashlock is not able to identify any further vulnerabilities.

Our Methodology

Hashlock strives to maintain a transparent working process and to make our audits a collaborative effort. The objective of our security audits is to improve the quality of systems and upcoming projects we review and to aim for sufficient remediation to help protect users and project leaders. Below is the methodology we use in our security audit process.

Manual Code Review:

In manually analysing all of the code, we seek to find any potential issues with code logic, error handling, protocol and header parsing, cryptographic errors, and random number generators. We also watch for areas where more defensive programming could reduce the risk of future mistakes and speed up future audits. Although our primary focus is on the in-scope code, we examine dependency code and behaviour when it is relevant to a particular line of investigation.

Vulnerability Analysis:

Our methodologies include manual code analysis, user interface interaction, and white box penetration testing. We consider the project's website, specifications, and whitepaper (if available) to attain a high-level understanding of what functionality the smart contract under review contains. We then communicate with the developers and founders to gain insight into their vision for the project. We install and deploy the relevant software, exploring the user interactions and roles. While we do this, we brainstorm threat models and attack surfaces. We read design documentation, review other audit results, search for similar projects, examine source code dependencies, skim open issue tickets, and generally investigate details other than the implementation.

Documenting Results:

We undergo a robust, transparent process for analysing potential security vulnerabilities and seeing them through to successful remediation. When a potential issue is discovered, we immediately create an issue entry for it in this document, even though we have not yet verified the feasibility and impact of the issue. This process is vast because we document our suspicions early even if they are later shown to not represent exploitable vulnerabilities. We generally follow a process of first documenting the suspicion with unresolved questions, and then confirming the issue through code analysis, live experimentation, or automated tests. Code analysis is the most tentative, and we strive to provide test code, log captures, or screenshots demonstrating our confirmation. After this, we analyse the feasibility of an attack in a live system.

Suggested Solutions:

We search for immediate mitigations that live deployments can take and finally, we suggest the requirements for remediation engineering for future releases. The mitigation and remediation recommendations should be scrutinised by the developers and deployment engineers, and successful mitigation and remediation is an ongoing collaborative process after we deliver our report, and before the contract details are made public.

Disclaimers

Hashlock's Disclaimer

Hashlock's team has analysed these smart contracts in accordance with the best industry practices at the date of this report, in relation to: cybersecurity vulnerabilities and issues in the smart contract source code, the details of which are disclosed in this report, (Source Code); the Source Code compilation, deployment, and functionality (performing the intended functions).

Due to the fact that the total number of test cases is unlimited, the audit makes no statements or warranties on the security of the code. It also cannot be considered as a sufficient assessment regarding the utility and safety of the code, bug-free status, or any other statements of the contract. While we have done our best in conducting the analysis and producing this report, it is important to note that you should not rely on this report only. We also suggest conducting a bug bounty program to confirm the high level of security of this smart contract.

Hashlock is not responsible for the safety of any funds and is not in any way liable for the security of the project.

Technical Disclaimer

Smart contracts are deployed and executed on a blockchain platform. The platform, its programming language, and other software related to the smart contract can have their own vulnerabilities that can lead to attacks. Thus, the audit can't guarantee the explicit security of the audited smart contracts.

About Hashlock

Hashlock is an Australian-based company aiming to help facilitate the successful widespread adoption of distributed ledger technology. Our key services all have a focus on security, as well as projects that focus on streamlined adoption in the business sector.

Hashlock is excited to continue to grow its partnerships with developers and other web3-oriented companies to collaborate on secure innovation, helping businesses and decentralised entities alike.

Website: hashlock.com.au

Contact: info@hashlock.com.au



#hashlock.